results: for example, we can specify that every page consists of 100 documents instead of the default 10 documents that Elasticsearch returns. There's also a function to highlight results with the search match words in the results. We can even ask the engine to grab results only from a specific set of shards using the shard routing function.

Additionally, most search queries support a handful of crosscutting functions, regardless of the type of query (term-level, full-text, geospatial query, etc.). Some are irrelevant to a specific type of query; for example, sorting isn't advisable on `text` fields, so it is limited to term-level queries. In the next few sections, we explore these features to understand their application in detail.

8.8.1 Pagination

More often than not, queries yields many results, possibly hundreds or even thousands. Sending all the results for a query simultaneously is asking for trouble because both the server side and client side need enough memory and processing capacity to deal with the data load.

By default, Elasticsearch sends the top 10 results, but we can change this number by setting the `size` parameter on the query. The maximum value is 10,000, but we can also change this limit, as discussed shortly. The query in the next listing sets `size` to `20`, returning the top 20 results at once.

Listing 8.17 Query to fetch a specific number of results

```
GET movies/_search
{
  "size": 20,
  "query": {
    "match_all": {}
  }
}
```

Setting `size` to `20` returns the top 20 results. If we have an index with 1 million documents, setting size to 10,000 retrieves that many documents (ignore the performance considerations for a moment!).

Resetting the 10,000-document size limit

The maximum number of results we can fetch by setting the `size` attribute is 10,000. Suppose we set the size to, say, `10001` and execute the following query:

```
GET movies/_search
{
  "size": 10001,
  "query": {
    "match_all": {}
  }
}
```